

Path Traversal vulnerability, is when a parameter loads a file from a directory and there's no security measures in place; Avoid passing raw user input to files-system. And instead map requests to a predefined list. Using Portswigger labs. We see that the website loads images of products from a directory, I tried to roll back to the root directory and read the /etc/passwd file. Which worked

The screenshot shows the Web Security Academy lab interface on the left and the Burp Suite interface on the right. The lab title is "File path traversal, simple case" and it is marked as "Solved". The lab description says "Congratulations, you solved the lab!". The Burp Suite interface shows a list of HTTP requests. The selected request is a GET request to `https://0a6400f04ee9378816a03a800f30013.web-security-academy.net/image?filename=48.jpg`. The request body is highlighted with a green circle, showing the payload `../../../../etc/passwd`. The Inspector panel on the right shows the request details, including the request body parameters.

I tried another lab, this lab had the same vulnerable parameter. The application blocked traversal sequences(`./` `../`). but the backend assumes the file supplied lives inside a specific asset folder. It concatenates the base path with the user's input, so I only typed `/etc/passwd`

The screenshot shows the Web Security Academy lab interface on the left and the Burp Suite interface on the right. The lab title is "File path traversal, traversal sequences blocked with absolute path bypass" and it is marked as "Solved". The lab description says "Congratulations, you solved the lab!". The Burp Suite interface shows a list of HTTP requests. The selected request is a GET request to `https://0a6400f04ee9378816a03a800f30013.web-security-academy.net/image?filename=13.jpg`. The request body is highlighted with a green circle, showing the payload `/etc/passwd`. The Inspector panel on the right shows the request details, including the request body parameters.